



MATURITNÍ PRÁCE

Vědomostní hra

Sára Jirkalová

vedoucí práce: Dr.rer.nat. Kočer Michal

Prohlášení

Prohlašuji, že jsem tuto práci vypracovala samostatně s vyznačením všech použitých pramenů a za pomoci jazykového modelu GPT-5.2, využitého pro generování otázek a doplnění komentářů do zdrojového kódu.

V Českých Budějovicích dne podpis

Sára Jirkalová

Abstrakt

Tato práce se zabývá návržením a implementací vědomostních her. V první části práce se čtenář seznámí s pojmem vědomostní hra a s několika příklady takových her na současném trhu. Dále se seznámí s moderním postupem při vývoji aplikací, s tzv. agilními metodami. Druhá část práce se zabývá návrhem a programováním nové vědomostní hry za využití těchto agilních metod.

Klíčová slova

Vědomostní hra, Agilní metody, Extrémní programování, Scrum, Python, Pygame

Poděkování

Děkuji především vedoucímu této maturitní práce, Dr.rer.nat. Michalu Kočerovi za trpělivost a za zodpovídání mých dotazů. Dále děkuji své rodině a svým přátelům za podporu při psaní této práce.

Obsah

I	Vědomostní hry v současnosti	2
1	Vědomostní hra	3
1.1	Dobyvatel	3
1.1.1	Historie	4
1.1.2	Průběh hry	5
1.2	Sporcle	6
1.2.1	Historie	6
1.3	Kahoot!	8
1.3.1	Historie	8
1.3.2	Průběh hry	9
1.3.3	Vliv na vzdělávání	9
2	Agilní metody vývoje softwaru	11
2.1	Historie	12
2.2	Extrémní programování	12
2.3	Testováním řízený vývoj	14
2.4	Scrum	15
II	Tvorba vědomostní počítačové hry	16
3	Cíl projektu	17
3.0.1	Agilní metody v této práci	17
3.1	Původní návrh	18

4 První verze	19
4.1 Verze 1.1	21
5 Druhá verze	23
5.1 Verze 2.1 Přidání souboje	24
5.2 Verze 2.2 Přidání mapy	25
5.3 Verze 2.3 Přidání obchodu	26
5.4 Verze 2.4 Přidání možnosti vložit otázku	26
5.5 Verze 2.5 Rozdělení otázek tvorům podle témat	27
6 Finální podoba hry	28
6.1 Blokové schéma	28
6.2 Průběh hry	29
6.3 Grafické rozhraní	31
Bibliografie	37
Přílohy	39

Úvod

Vědomostní hry se v dnešní době hrají stále častěji a to nejen ve volném čase, ale jsou také využívány ve školství pro obohacení výuky. Proto je cílem této práce seznámit čtenáře s aktuální scénou vědomostních her i jejich využití ve školství.

První část práce se zaměří na současný stav vědomostních her. Konkrétně se bude zabývat hrami *Dobyvatel*, *Sporcle* a *Kahoot!*, jejich historií i současnou podobou.

Dále se práce bude zabývat agilními metodami. Přiblíží čtenáři pojem *Agilní vývoj softwaru* a představí mu některé takové metody a jejich přínosy při vývoji softwaru. Těmito metodami budou *Extrémní programování*, *Testováním řízený vývoj* a *Scrum*.

V hlavní části práce se pokusím vyvinout svou vlastní vědomostní hru. Budu se přitom snažit co nejvíce zásadami agilního vývoje. Pro psaní kódu využiji programovací jazyk *Python* a jeho rozšíření *Pygame*. V práci poté popíši postup vývoje této hry a její konečnou podobu.

Část I

Vědomostní hry v současnosti

1 Vědomostní hra

Pojem vědomostní hra v této maturitní práci označuje hru založenou na odpovídání na otázky na různá témata, např. geografie, biologie, jazyky, či všeobecný přehled. Vědomostní hry lze zařadit mezi tzv. didaktické hry. To jsou takové hry, jejichž primární cíl není zábava, ale vzdělávání. Tyto hry se snaží zlepšit a zefektivnit učení za pomoci prvků herního designu, např. systém výzev a odměn, zpětná vazba atd. [4]

Videohry jako nástroj pro učení se využívají hlavně proto, že zvyšují motivaci studentů a tím zvyšují i čas, který stráví učením se [15]. Pokud jsou tyto hry navrženy správně, dokáží přinášet lepší výsledky než tradiční způsoby učení. Nicméně, vědomostní a didaktické hry často nedokáží být dostatečně efektivní v zapojení a motivování studentů a vzbuzování jejich zájmu. [4]

1.1 Dobyvatel

Hra Dobyvatel je českou verzí maďarské vědomostní a strategické online hry *ConQUIZtador*, vyvíjenou společností *THX Games* [25]. V této hře proti sobě bojují tři hráči o jednotlivá území České republiky odpovídáním na dva typy otázek (*otázky s výběrem správné odpovědi* a *tipovací otázky*). Když hráč odpoví správně, nebo lépe než ostatní hráči v případě *tipovacích otázek*, má možnost zabrat nějaké území. Za každé získané území dostane hráč body. Vyhrává hráč s nejvyšším počtem bodů. [29]

Odpovědí na *tipovací otázku* je vždy celé nezáporné číslo a vyhrává ten hráč, kterého odpověď byla nejbližší té správné. Pokud je několik odpovědí stejně daleko, vyhrává ten hráč, který odpověděl jako první. U *otázek s výběrem správné odpovědi* mají hráči vždy na výběr ze čtyř možných odpovědí, z nichž je vždy právě jedna správná. [30]

1.1.1 Historie

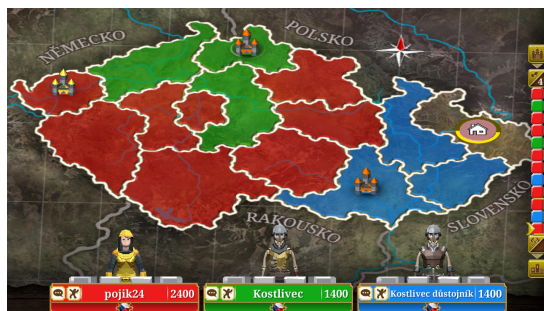
Hra *Dobyvatel* byla spuštěna v roce 2010 na portálu *TV Nova*. Již během prvních dvou let si hru vyzkoušelo více než 500 000 lidí [44]. V roce 2016 byla změněna grafická úprava hry a byly i pozměněny některé herní mechaniky. Tato změna se však nesetkala s velkým úspěchem v hráčské komunitě, někteří hráči si stěžovali na dětinský vzhled nové verze, nepřehlednost uživatelského rozhraní a na mikrotransakce [10]. Vznikla dokonce petice za obnovení starého *Dobyvatele*. Tato petice však nezískala ani tisíc podpisů [14].



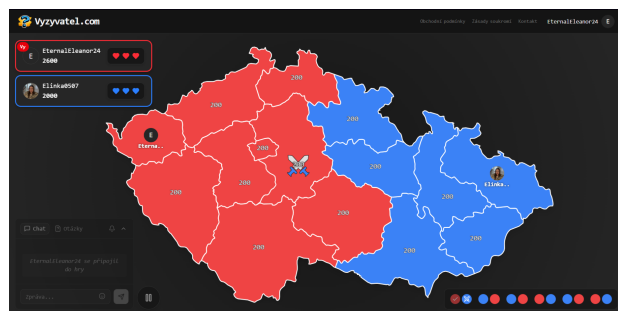
Obrázek 1.1: Ukázka GUI „starého“ *Dobyvatele* [13] Obrázek 1.2: Ukázka GUI „nového“ *Dobyvatele* [27]

„Nový *Dobyvatel*“, verze od roku 2016, přetrval až do roku 2021. Na konci tohoto roku byla totiž webová verze hry *Dobyvatel* zrušena kvůli ukončení podpory *Adobe Flash Player* [41]. Dnes je možné si hru zahrát pouze na mobilních zřízeních. Tato mobilní verze *Dobyvatele* však nebyla mezi hráči příliš populární, v roce 2023 měla na *Google Play* hodnocení pouze 2,3 hvězdiček z pěti [11]. V současnosti má ale už hodnocení 4,3 hvězdiček [12]. 25 000 negativních hodnocení bylo totiž smazáno. Mohlo se jednat o spam nebo falešné recenze [17]. Mobilní verze *Dobyvatele* je nejčastěji kritizována za vzhled a nedostatek času na označení odpovědi [12].

Nejen kvůli této kritice začaly vznikat různé fanouškovské projekty, které se snaží simulovat původní webovou aplikaci. Jedním z takových projektů je hra *Vyzyvatel*. Autor, vlastním jménem Maroš Mečiar, někdy vystupující pod přezdívkou *Maroso*, se svou hrou snažil nejen přiblížit původnímu dobyvateli, ale i přidat něco navíc. Například si hráči mohou vybrat pouze sady otázek z určitých oborů, nebo si dokonce napsat vlastní. [28]



Obrázek 1.3: Ukázka GUI mobilní aplikace *Dobyvatel* [12]



Obrázek 1.4: Ukázka GUI hry *Vyzyvateľ* [28]

1.1.2 Průběh hry

Průběh hry se liší v závislosti na tom, zda se jedná o *Běžnou hru*, nebo *Dlouhou kampaň*. *Běžná hra* má tři fáze. Při první fázi, s názvem *Stavba základny*, hra náhodně přidělí každému hráči základnu na jednom z polí. Tato pole jsou vybrána tak, aby se nedotýkala. Pole se základnou má hodnotu 1000 bodů a obsahuje tři věže. Druhá fáze se jmenuje *Expanze*. Při této fázi dostávají všichni hráči tipovací otázku. Vítězný hráč si může vybrat dvě pole, hráč který se umístil druhý jedno pole. Hráči si mohou vybrat pouze prázdná pole, a pokud takové existují, dotýkající se jejich základny nebo jejich jiného pole. Hráč dostane 200 bodů za každé takto zabrané území. [40]

Třetí a poslední fáze s názvem *Bitva* se skládá ze čtyř částí, neboli kol, při kterých hráči postupně útočí na nepřátelská území. Pořadí hráčů je v každém kole, vyjma posledního, jiné. V posledním kole je pořadí určeno počtem bodů hráčů. Kdo má více bodů, útočí dříve. Útočník a hráč na kterého je útočeno (obránce) odpovídají na otázku s jednou správnou odpovědí. Třetí hráč je pozoruje, ale jinak se boje neúčastní. Pokud útočník neodpověděl správně, nebo na otázku nereagoval, vyhrává obránce. Pokud útočník zvolí správnou odpověď a obránce zvolí špatnou, vyhrává útočník a pokud odpoví oba hráči správně, rozhodne se bitva tipovací otázkou. Pokud útočník vyhraje, získá pole i s body, a bodová hodnota tohoto pole se nastaví na 400. Obránce tím pádem přichází nejen o území, ale taky o body, které tomuto území náležely. Pokud se ale obránci podaří území ubránit správným zodpovězením otázky, získá defenzivní bonus 100 bodů. [40]

Při útoku na základnu musí útočník rozbít všechny její věže, to znamená, že musí ve svém tahu zodpovědět jednu otázku za každou věž. Pokud se obránci podaří úspěšně ubránit,

bitva končí, ale všechny již dobyté věže zůstanou dobyté a při dalším pokusu je útočník nemusí dobývat znova. Když padne poslední věž, obránce ztrácí všechno své území a body ve prospěch útočníka. Pokud poslední bitva nerozhodne o výherci celé hry, dostanou hráči ještě jednu tipovací otázku za 10 bodů. [40]

Dlouhá kampaň je rozdělena do čtyř fází. První se nazývá *Stavba základny*. Během této fáze si na rozdíl od *Běžné hry* hráči vybírají umístění základny sami. Nejdříve si vybírá červený hráč, poté modrý a nakonec zelený. Barvy jsou hráčům přiděleny náhodně. Stejně jako u *Běžné hry* se základny nemohou dotýkat a každá má hodnotu 1000 bodů. Následuje fáze s názvem *Expanze*. Zde si hráči vybírají sousední území a mohou je obsadit, pokud správně odpoví na otázku s výběrem správné odpovědi. Takto získaná území mají hodnotu 200 bodů. Tato fáze má šest kol a pořadí hráčů se mění v každém kole. Třetí fáze se jmenuje *Rozdělení volných území*. V této fázi dostávají hráči tipovací otázky a výherce každé této otázky si může zabrat jedno volné území. Tato pole mají hodnocení 300 bodů. Fáze končí obsazením posledního volného území. Čtvrtá fáze, *Bitva*, probíhá téměř stejně jako u *Běžné hry*. Jediným rozdílem je to, že si hráč může dostavět padlé věže své základny zodpovězením otázky s výběrem správné odpovědi. [30]

1.2 Sporcle

Sporcle je webová a mobilní aplikace umožňující lidem vytvářet a hrát vědomostní kvízy [1]. Tyto kvízy jsou rozděleny do 15 hlavních kategorií: sporty, geografie, hudba, filmy, televize, jen pro zábavu, historie, literatura, jazyk, věda, gaming, zábava, náboženství, svátky a ostatní. *Sporcle* také nabízí několik formátů kvízů, např. klasický, kdy musí hráč napsat správnou odpověď, klikací, s mapu, atd. Existuje také několik módů, které může autor kvízu nastavit, například minové pole, což znamená, že se kvíz automaticky ukončí po zadání špatné odpovědi nebo pevné pořadí, hráč nemůže přeskakovat mezi otázkami, atd. [39]

1.2.1 Historie

Matt Ramme a Ali Aydar založili Sporcle v lednu roku 2007. Původně to byla stránka, na které mohli uživatelé zkoušet předvídat sportovní zápasy. Tehdy také vznikl název *Sporcle* jako složenina anglických slov *sport* a *oracle* (česky: sport a orákulum). Tato skutečnost se

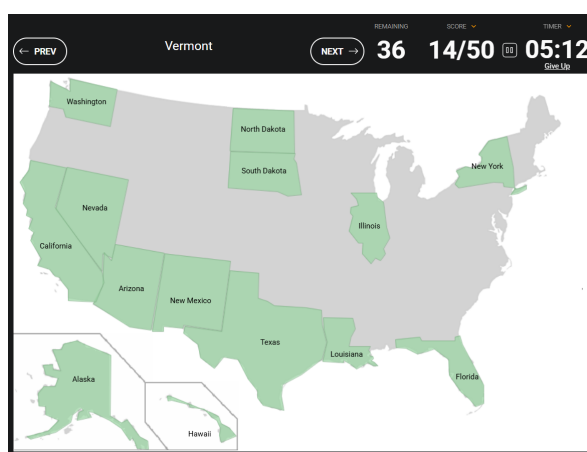
také podepsala na vzhedu loga společnosti, na němž je křišťálová koule. [35]



Obrázek 1.5: Současné logo *Sporcle*[38]

Změna povahy této stránky ale nastala ještě v tomtéž roce. Matt Ramme totiž hledal jiné stránky, na kterých by mohl otestovat a doplnit své znalosti, ale zjistil, že jich moc nebylo. Tak se rozhodl začít kvízy přidávat na *Sporcle*. První kvíz, s názvem: „Vyjmenuj prezidenty USA“, přidal v létě roku 2007. Již druhý den od vydání se tento kvíz dostal na přední stránku sociálního zpravodajského webu *digg.com* [20]. V jednom dni si tento kvíz zahrálo přes sto tisíc lidí. Zájem o tyto kvízy poté nadále rostl a tak Ramme přidal uživatelům možnost si vytvořit vlastní kvízy a tyto kvízy také hodnotit. [35]

Navzdory autorovu prvotnímu očekávání se web uchytil i mezi studenty. [5] Aby tento zájem ještě více vývojáři podpořili, přidali v listopadu roku 2010 žebříček univerzit podle toho, kolik jejich studentů stránku navštívilo.[49] V roce 2010 vyvinul Bob Scheld, bývalý zaměstnavatel Matta Ramme, také mobilní aplikaci *Sporcle* [35].



Obrázek 1.6: Najdi státy USA – bez obrysů, minové pole[36]

Už v roce 2013 *Sporcle* přesáhlo jednu miliardu zahraničních kvízů a v roce 2014 přesáhlo 500 000 vytvořených kvízů. V současnosti *Sporcle* dosahuje asi milion zahraničních kvízů denně a téměř šest a půl miliardy zahrání celkem [1]. Nejhranější kvíz, s osmdesáti miliony přehrání,

se jmenuje „Find the US States - No Outlines Minefield“, [37] v překladu „Najdi státy USA – bez obrysů, minové pole“. Minové pole v názvu znamená, že pokud hráč udělá chybu, celý kvíz se ukončí. [36]

1.3 Kahoot!

Kahoot! je Norská online vzdělávací platforma umožňující lidem tvořit a hrát kvízy založené na otázkách s výběrem správné odpovědi [22]. Původní prototyp této platformy byl vytvořen už v roce 2006, ovšem k největšímu rozmachu došlo až v roce 2020 kvůli pandemii COVID-19 [47][19].

Cílem této platformy je zvýšení aktivního zapojení, motivace a soustředění žáků a studentů a zlepšit jejich studijní výsledky. Aplikace *Kahoot!* je také používána jako nástroj formativního hodnocení nebo jako nahrazení tradičních výukových prostředků. [48]

1.3.1 Historie

Původní prototyp který vedl až ke spuštění platformy *Kahoot!* se nazýval *Lecture Quiz*. Byla to mobilní aplikace a vyvíjeli ji Alf Inge Wang, Terje Øfsdahl a Ole Kristian Mørch-Storstein na katedře informatiky Norské univerzity vědy a technologie. Tato aplikace měla za úkol lépe zapojit studenty do výuky. Poprvé byla použita v roce 2007 na univerzitě, kde byla vyvíjena, na přednášce a získala od studentů pozitivní recenze. Studenti uvedli, že se více soustředili na přednášku a hra jim přišla přínosná pro ně i pro profesora. Hra však měla pár technických problémů, které se ale podařilo vyřešit. [47]

Druhá verze s názvem *Lecture Quiz 2.0* vyšla v roce 2011 na téže univerzitě jako verze předchozí. Autoři projektu se u druhé verze zaměřili zejména na stabilitu celého systému, zjednodušení používání pro studenty i učitele a dobrou dokumentaci usnadňující další vývoj platformy. Těchto cílů chtěli dosáhnout vytvořením webového rozhraní. Testu této vylepšené hry se zúčastnilo 21 studentů v rámci výuky a test tentokrát proběhl téměř bez technických závad. Z dotazníkového šetření uskutečněného po testu vyplývá, že cíle tohoto projektu byly dosaženy. [50]

Morten Versvik, Johan Brand, and Jamie Brooker založili společnost *Kahoot!* v roce 2012. Technologie této společnosti je založena na výzkumu, jenž provedl Morten Versvik, jehož učil Alf Inge Wang, který se také zapojil do vývoje této vědomostní hry. V roce 2013 byla tato platforma poprvé zpřístupněna pro veřejnost [22]. Už v roce 2017 měla tato hra přes 40 milionů aktivních hráčů měsíčně a tento rok také společnost *Kahoot!* vydala mobilní aplikaci, skrze kterou mohou učitelé zadávat žákům domácí úkoly [24][33].

1.3.2 Průběh hry

Jednotlivé kvízy se nejčastěji hrají pomocí jedné velké obrazovky a mobilních telefonů pro každého hráče. Na velké obrazovce se zobrazí otázka a výběr odpovědí a hráči pomocí mobilních zařízení odpovídají. Každý hráč hraje pod zvolenou přezdívku, takže kvíz může být anonymní, což pomáhá snižovat strach ze špatné odpovědi. Za každou správnou odpověď dostanou hráči počet bodů podle toho, jak rychle odpověděli. Mezi otázkami se vždy zobrazí na velké obrazovce správná odpověď a distribuce odpovědí, což dává zpětnou vazbu tomu, kdo kvíz vytvořil. Například, pokud většina hráčů zadala stejnou špatnou odpověď, mohla být otázka položena nepřesně. Poté se na obrazovce zobrazí přezdívky a body pěti nejúspěšnějších hráčů. Zároveň každý hráč vidí individuální zpětnou vazbu na své obrazovce. Hráč s největším počtem bodů vyhrává. Tento styl hry se dá využívat například ve školách, ve firmách nebo na různých společenských akcích. [45]

Druhá možnost jak hrát *Kahoot!* nepotřebuje velkou obrazovku. Každý hráč na svém zařízení vidí otázku i možné odpovědi a kvíz si může zahrát kdekoli a kdykoli. Tuto možnost je vhodné používat například na domácí úkoly, školení, atp. [23]

1.3.3 Vliv na vzdělávání

Nedostatek motivace mezi žáky může vyústit ve snížení efektivity vzdělání a negativní atmosféru ve třídě. Tento problém je ještě zřetelnější na vysokých školách s velkými třídami a málo interakce mezi přednášejícím a studenty. Výzkum ukázal, že studenti, kteří jsou aktivně zapojeni do výuky se naučí více, než pasivní studenti. Jedním ze způsobů jak žáky a studenty do výuky více zapojit jsou systémy pro odpovědi studentů (SRS — z anglického „Student response systems“). Dalším způsobem jak motivaci žáků a studentů zvýšit, je učení pomocí her. *Kahoot!* je kombinací systému pro odpovědi studentů a videohry. [48]

Většina studií porovnávající efekt aplikace *Kahoot!* a jiných výukových prostředků na výsledky vzdělávání se shoduje v tom, že používání hry *Kahoot!* při výuce způsobuje zlepšení výsledků vzdělávání oproti nejen tradičním výukovým nástrojům, ale i oproti jinému systému pro odpovědi studentů, který neměl herní prvky. Studenti kteří používali *Kahoot!* se také cítili více aktivně zapojeni do výuky a byli méně ve stresu. [42]

Další studie se zabývala rozdíly mezi dlouhodobým a krátkodobým používáním hry *Kahoot!*. Porovnávala názory studentů na několik otázek poté, co hráli *Kahoot!* v jedné přednášce s názory na téže otázky po používání *Kahootu* během celého semestru. Tato studie zjistila, že největší rozdíl byl v dynamice ve třídě. Studenti, kteří hráli *Kahoot!* jednou spolu během hry více komunikovali. U ostatních otázek byly rozdíly minimální. [45]

Jedna studie se také věnovala vlivu hudby a bodování, které jsou oboje součástí aplikace a mají za cíl zvýšit zájem a soutěživost mezi studenty. Studie se zaměřovala na motivaci, koncentraci a zájem studentů a dynamiku ve třídě. Bylo zjištěno, že přítomnost hudby měla vliv na celkovou energii studentů. Když byla hudba zapnutá, studenti více komunikovali, diskutovali a byli otevření dotazům vyučujícího. Bodování zvyšovalo motivaci studentů. Na druhou stranu měli však studenti bez bodování častěji pocit, že se naučili něco nového. [46]

2 Agilní metody vývoje softwaru

Agilní metody vývoje softwaru jsou takové metody, které následují hodnoty *Manifestu Agilního vývoje software* a jeho 12 principů. Hlavními body *Agilního manifestu* jsou:

- Jednotlivci a interakce před procesy a nástroji
- Fungující software před vyčerpávající dokumentací
- Spolupráce se zákazníkem před vyjednáváním o smlouvě
- Reagování na změny před dodržováním plánu [8]

První bod poukazuje na jeden z nejdůležitějších rozdílů mezi agilním a jinými přístupy k vývoji softwaru, kterým je zaměření na lidi, jejich interakce mezi sebou a jejich spolupráci. Druhý bod manifestu říká, že i když je dokumentace důležitá, hlavní prioritou vývojářů by mělo být dodávat funkční software. Při agilním vývoji softwaru je nutná neustálá komunikace se zákazníkem a tuto skutečnost odráží třetí bod manifestu. Tato spolupráce se zákazníkem může často vést k změnám v požadavcích na vyvíjený software. Změna je pro agilní vývoj softwaru velice důležitá, jak dokazuje poslední bod manifestu. Vývojáři pracující agilně tedy vítají změnu požadavků i během samotného vývoje. I proto zvolili autoři název *Agilní*, protože odráží důležitost přizpůsobivosti a připravenosti na změnu. [3]

Dvanáct principů agilního vývoje softwaru dávají jasnější pokyny k zavedení agilního postupu. Tyto principy vyzdvihují individualitu, neustálou spolupráci se zákazníkem, samoorganizující se týmy, osobní konverzaci, často dodávaný jednoduchý software a důležitost změny. [9]

2.1 Historie

Za počátek agilního vývoje se většinou považuje schůze 17 vývojářů v Utahu v roce 2001. Agilní přístup ovšem existoval i předtím, jen nebyl ucelený a neměl zatím název. Už v polovině devadesátých let vymýšleli lidé nové postupy, které se podobaly dnešnímu agilnímu vývoji. *Manifest Agilního vývoje software* vznikl právě na již zmíněné schůzi v Utahu. Tento manifest shrnul základní myšlenku a poprvé zde byl užit termín *Agilní vývoj software*. V následujících měsících doplnili autoři manifest o 12 principů agilního vývoje software. Na konci roku 2001 byla také vytvořena *Agile Alliance*. Jejím cílem je spojovat lidi vyvíjející software, aby mohli sdílet nápady a zkušenosti. [2]

2.2 Extrémní programování

Jednou z agilních metod vývoje je tzv. Extrémní programování. Jedná se o metodiku pro malé až střední týmy, které vyvíjejí software a mají nejasné, nebo často se měnící zadání. Cíl extrémního programování je snižovat projektová rizika, zlepšit schopnost reagovat na změny a zvýšit produktivitu. [6]

Tato metodika je vlastně soubor několika postupů. Těmi postupy jsou:

- Plánovací hra (Rychle vytvoříme plán na další verzi, pokud je skutečnost jiná, plán zaktualizujeme)
- Malé verze (Rychle vytvoříme základní verzi projektu, kterou poté vytváříme další verze v krátkých cyklech)
- Metafora (Celý tým má stejnou představu o tom, jak má celý systém vypadat)
- Jednoduchý návrh (Systém by měl být navržen vždy co nejjednodušeji)
- Testování (Programátoři píší neustále testy jednotek, zákazník provádí testy funkcionality)
- Refaktorizace (Programátoři vylepšují program bez toho, aby změnili jeho funkcionality)
- Párové programování (Všechn kód by měl být psán v párech)

- Společné vlastnictví (Všichni programátoři nesou zodpovědnost za celý kód, každý může jakoukoli část kdykoli upravit)
 - Nepřetržitá integrace (Nový kód by se měl několikrát denně integrovat do stávajícího),
 - 40-hodinový pracovní týden (Pracovní týden by neměl překročit 40 hodin a pokud je zapotřebí pracovat přesčas, nikdy takto nepracujeme dva týdny za sebou)
 - Zákazník na pracovišti (zákazník by měl mít na pracovišti zástupce, kterému můžou programátoři neustále klást dotazy a který může provádět testy funkcionality)
 - Standarty pro psaní zdrojového kódu (Zdrojový kód by měl mít jednotnou strukturu)
- [6]

Jedním z kontroverznějších z těchto postupů je bezpochyby párové programování. Jedná se o postup, při kterém programují dva lidé zároveň na jednom počítači, s jednou klávesnicí a jednou myší. Celý proces spočívá v tom, že vždy jeden z programátorů píše a druhý ho kontroluje a zároveň přemýšlí nad dalším problémem, dalším testem nebo možnou refaktORIZACÍ. Výhodou oproti tradičnímu postupu je například fakt, že při párovém programování se více chyb odhalí už při samotném psaní kódu. Další výhodou je i to, že když člověk programuje s někým, tak bude nejspíše produktivnější, protože se z úcty ke druhému nenechá rozptýlit telefonátem nebo jinou soukromou záležitostí. To nás ale přivádí k jedné z nevýhod a tou je to, že tento proces funguje pouze za předpokladu, že se tito dva lidé vzájemně snesou a mají k sobě dostatek respektu. [6]

Podle metaanalýzy z roku 2009 nelze jednoznačně určit, zda je párové programování lepší než sólo programování, ale vždy záleží na dané situaci. Párové programování je totiž užitečnější, pokud potřebujeme jednoduchý úkol splnit co nejrychleji, i za cenu ztracené kvality, nebo potřebujeme splnit složitý úkol kvalitněji, i za vyšší cenu [18]. Párové programování také samozřejmě nebude lepší, pokud se členové týmu nerespektují, nebo se nesnesou. V takovém případě je nutné, aby spolu tito lidé nepracovali a někdy toto může vést i k tomu, že takový člověk musí z týmu odejít[6].

I přesto, že párové programování nemusí být vždy lepší, mají vývojáři na extrémní programování pozitivní názory. Většina takto pracujících vývojářů by u tohoto postupu chtěla

zůstat. Tito vývojáři mají také vyšší uspokojení z práce a věří, že jsou produktivnější. Podobné názory převládají také mezi studenty. Většina z dotázaných věřila, že extrémní programování zlepšuje kvalitu kódu a tento postup by doporučila svým budoucím zaměstnavatelům. [16]

Zda extrémní programování opravdu zvyšuje produktivitu vývojářského týmu se ovšem nedá jednoznačně určit. Podle jedné studie se produktivita, měřena počtem napsaných řádků kódu, zvedla o 46 %, ale postupně s časem začala klesat zpět na stejné hodnoty, jaké měl tým využívající tradiční metody[21]. Oproti tomu jiná studie zjistila 44% pokles v produktivitě při používání extrémního programování [16].

2.3 Testováním řízený vývoj

Testováním řízený vývoj je metoda vývoje softwaru, která se snaží o *čistý kód, který funguje*. Tohoto se snaží dosáhnout dvěma pravidly: Pište nový kód, pouze pokud selhal automatický test a Eliminujte duplicitu. Tyto dvě pravidla určují postup při psaní nového kódu:

1. Napište malý test, který ze začátku nefunguje
2. Rychle zaříd'te, aby test fungoval i za cenu špatného kódu
3. Refaktorizujte—odeberte duplicitu způsobenou předchozím krokem

Důvodem pro psaní testů je zamezení strachu. Kvůli strachu totiž můžeme být nerozhodní, málo komunikovat a bát se zpětné vazby. Právě automatizované testy nám ale mohou pomoci se méně bát. Víme totiž, že pokud všechny testy fungují, pak i celý kód bude fungovat a pokud nějaký fungovat přestane, hned víme, kde máme hledat chybu. Další způsob, jak tyto testy eliminují strach je to, že nám pomohou si větší úkol rozdělit do mnoha malých. [7]

Podle systematického přehledu z roku 2010, který zkoumal 32 různých studií se zaměřením na efektivitu *Testováním řízeného vývoje*, se nedá jednoznačně určit, zda je tato metoda vývoje softwaru lepší, než klasické postupy. Celková kvalita softwaru ani produktivita nebyla jednoznačně lepší, nebo horší než při využití jiných postupů. Jediná oblast, ve které byla tato metoda lepší, je kvalita testů, ale ani ta nebyla o tolik vyšší, než kolik se předpokládalo.

Ve výsledku tedy opět záleží na jednotlivém týmu, zda bude tento postup efektivnější, či nikoli. [43]

2.4 Scrum

Scrum je odlehčený rámec založený na krátkých časových intervalech, tzv. *Sprintech* a na *Scrum týmu*. *Scrum tým* se skládá z vývojářů, *Vlastníka produktu* a z *Mistra Scrumu*. *Vlastník produktu* se snaží maximalizovat hodnotu výsledného produktu. *Mistr Scrumu* dohlíží na dodržování hodnot a postupů v rámci *Scrum*. Pomáhá také každému členu týmu *Scrumu* porozumět. [34]

Sprint je označení pro vývojový cyklus *Scrum týmu*. Jeden *Sprint* by měl trvat maximálně jeden měsíc, ale mohou trvat i kratší dobu. Na začátku každého *Sprintu* by mělo proběhnout *Plánování Sprintu*. Během tohoto plánování *Vlastník produktu* navrhuje, jak v tomto *Sprintu* zvýšit hodnotu produktu a společně s celým týmem sestavuje *Cíl Sprintu*. Vývojáři vyberou úkoly, které budou zahrnuty do *Sprintu*. Celé plánování by nemělo trvat déle než osm hodin pro *Sprint* o délce jednoho měsíce. Další součástí *Sprintu* je *Denní Scrum*, což je patnácti minutová schůze *Scrum týmu*, konaná každý den ve stejný čas a na stejném místě. Na této schůzi vývojáři vytvoří plán pro daný den. *Sprint* končí *Revizí Sprintu*, jejímž úkolem je prezentace výsledků daného *Sprintu* a diskuze mezi *Scrum týmem* a důležitými zúčastněnými stranami. Tato revize by neměla trvat déle než čtyři hodiny pro *Sprint* o délce jednoho měsíce. [34]

Podle studie z roku 2005 jsou názory na zavedení *Scrumu* pozitivní mezi vývojáři i mezi zákazníky. Všichni dotázaní vývojáři by tento rámec doporučili i v budoucnu. Po zavedení *Scrumu* byly vývojáři spokojenější s odvedenou prací a byli si jistější, že vytvářejí produkt, se kterým bude spokojen i zákazník. Zákazníci také byly spokojenější s výsledným produktem a také by v budoucnu *Scrum* doporučili. Zákazníci začali být také více zaujatí prací vývojářů, protože se stali součástí vývoje, a také k nim začali mít větší respekt. Mimoto se se zavedením *Scrumu* snížil počet odpracovaných přesčasů, což vede ke udržitelnějšímu vývoji. [26]

Část II

Tvorba vědomostní počítačové hry

3 Cíl projektu

Cílem tohoto projektu bylo naprogramovat vědomostní hru a použít přitom moderní postupy agilního programování. Tato hra by měla hráče dostatečně motivovat, aby byl efekt učení co nejvyšší. Dále by měl hráč mít možnost si přidat vlastní otázky a procvičovat cíleně témata, která se potřebuje naučit, nebo která ho zajímají. Dále by měla hra být pochopitelná i pro lidi, kteří nikdy počítačové hry nehráli.

Na vytvoření tohoto programu jsem se rozhodla použít programovací jazyk *Python* s rozšířením *Pygame*. Je to set modulů navržený pro tvorbu her a multimediálních programů [31]. Pro jeho instalaci je potřeba mít nainstalovaný *Python*. *Pygame* instalujeme pomocí instalačního programu balíčků pro *Python* (zkráceně pip)[32].

3.0.1 Agilní metody v této práci

Pro svou práci jsem se rozhodla použít extrémní programování, protože má jasně dané postupy kterými jsem se mohla při psaní programu řídit. Vzhledem k povaze a velikosti projektu a vzhledem k tomu, že nepracuji v týmu, jsem se neřídila všemi postupy, ale vybrala jsem jen ty, které se k mojí práci hodí. I přesto, že jsem nepoužila všechny postupy jsem pořád postupovala agilně. Postupy, které jsem vybrala jsou: Plánovací hra, Malé verze, Metafora a Jednoduchý návrh.

Pracovala jsem tedy tak, že jsem nejdříve vytvořila velice jednoduchý prototyp mé hry a postupně jsem přidávala další funkcionalitu v malých verzích. Vždy když jsem jednu dokončila, udělala jsem plán té další.

3.1 Původní návrh

Jelikož jsem postupovala agilně, měla jsem ze začátku pouze hrubý návrh, který jsem postupně upřesňovala. Počáteční návrh byl vědomostní hra, ve které by hráč soutěžil s počítačem, nebo jiným hráčem. Hráč by dostával otázky z různých oborů a za správnou odpověď by dostal body. Vyhrál by ten, kdo by měl nejvíce bodů. Poté, co jsem dokončila demo takovéto hry, jsem se rozhodla, že hráč bude rytíř procházející mapu a pokaždé, když vejde na nové políčko, má nějakou šanci na to, že ho napadne nějaká příšera a začne souboj. Při souboji bude hráč dostávat otázky a vždy když správně odpoví, dá příšěře nějaké poškození, jestliže odpoví špatně, poškození naopak dostane od příšery. Za každou zabitou příšeru by hráč dostal pár peněz herní měny, za které by si potom mohl koupit předměty, které by mu zvýšily šanci na výhru.

4 První verze

V této verzi jsem vytvořila jednoduchý prototyp mé hry. Byla to pouze terminálová aplikace pro dva hráče, kteří se střídali v odpovídání na otázky a nakonec aplikace vyhodnotila, který hráč získal více bodů.

Začala jsem importováním modulu `csv`, který jsem použila pro manipulaci s databází otázek, která měla formu CSV souboru. Poté jsem nadefinovala třídu `Otazka`. Tato třída má čtyři atributy, a těmi jsou: `Typ`, `jenOtazka`, `spravnaOdpoved`, `vsechnyOdpovedi`. Její konstruktor má parametr `cisloOtazky`. Tento konstruktor vyhledá v tabulce otázku s požadovaným číslem a přiřadí atributům třídy odpovídající proměnné.

```
1 class Otazka(object):
2     def __init__(self, cisloOtazky):
3         with open("Otazky.csv", mode='r', encoding='utf-8') as
           soubor:
4             data = csv.DictReader(soubor, delimiter=";")
5             rows = list(data)
6             self.typ = rows[cisloOtazky]["Tema"]
7             self.jenOtazka = rows[cisloOtazky]["Otazka"]
8             self.spravnaOdpoved = rows[cisloOtazky]["
           SpravnaOdpoved"]
9             self.vsechnyOdpovedi = [rows[cisloOtazky]["
           SpravnaOdpoved"], rows[cisloOtazky]["
           SpatnaOdpoved1"], rows[cisloOtazky]["
           SpatnaOdpoved2"], rows[cisloOtazky]["
           SpatnaOdpoved3"]]
```

Zdrojový kód 4.1: Konstruktor třídy `Otazka`

Druhou třídou tohoto programu je třída `DemoHrac`. Má pouze dvě proměnné a těmi jsou `jmeno` a `skore`.

Třetí třídou tohoto programu je třída `Demo`. Má tři parametry: `pocetKol`, `jmenoHrace1`

a `jmenoHrace2`. Ve svém konstruktoru inicializuje dva objekty třídy `Hrac`, každý s jedním ze zadaných jmen.

Dále jsem nadefinovala metodu `zeptej` třídy `Demo`. Tato metoda má jeden parametr `otazka`, což je objekt třídy `Otazka`. Metoda `zeptej` nejdříve zobrazí hráči otázku a výběr odpovědi v náhodném pořadí. Pro uspořádání odpovědí do náhodného pořadí jsem použila modul `random` a jeho metodu `shuffle`. Program poté vyzve hráče k zadání správné odpovědi. Následně ověří správnost odpovědi pomocí metody `kontrola`, která vrací `True` pokud je odpověď správná, v opačném případě vrací `False`. Pokud hráč zadal správnou odpověď, metoda `zeptej` vrací `True` a vypíše na terminál „Správně!“. Pokud hráč zadal naopak špatnou odpověď, metoda vrátí `False` a vypíše zprávu „Špatně!“.

```
1 def zeptej(self, otazka):
2     ABCD = {"A":0, "B":1, "C":2, "D":3}
3     print(otazka.jenOtazka)
4     random.shuffle(otazka.vsechnyOdpovedi)
5     print(otazka.vsechnyOdpovedi)
6     while True:
7         print("Zadej jednu z možností A, B, C, D")
8         odpoved = input("Zadej odpověď: ")
9         if odpoved in ABCD:
10             break
11     odpoved = otazka.vsechnyOdpovedi[ABCD[odpoved]]
12     if otazka.kontrola(odpoved):
13         print("Správně!")
14         return True
15     else:
16         print("Špatně!")
17         return False
```

Zdrojový kód 4.2: Metoda `zeptej`

Další metodou třídy `Otazka` je `tah`. Má jeden parametr `hrac` (objekt třídy `Hrac`). Tato metoda vybere náhodnou otázku pomocí funkce `randint` modulu `random`. Poté zavolá metodu `zeptej` a pokud vrátí `True`, připsá hráči jeden bod.

```
1 def tah(self, hrac):
2     otazka = Otazka(random.randint(1,10))
3     spravnost = self.zeptej(otazka)
4     if spravnost:
```

```
5 | hrac.skore += 1
```

Zdrojový kód 4.3: Metoda `tah`

Poslední metodou třídy `Demo` je `run`. Tato metoda postupně zavolá funkci `tah` pro každého hráče a poté vytiskne aktuální skóre na terminál. Toto opakuje několikrát, podle toho, jaký počet kol byl zadán.

Nakonec program vytvoří instanci třídy `Demo` a spustí její metodu `run`.

```
1 | if __name__ == "__main__":
2 |     aplikace = Demo(3, "Lubomír", "Miroslav")
3 |     aplikace.run()
```

Zdrojový kód 4.4: Spuštění dema

4.1 Verze 1.1

Tato verze se od té první liší tím, že místo CSV souboru využívá SQL databázi. Pro práci s touto databází využívá modul `Databaze`. Tento modul má jednu třídu s názvem `DbOtazek`. V konstruktoru této třídy se vytvoří tabulka s názvem `otazky` a strukturou zobrazenou v tabulce 4.1.

Tabulka 4.1: Struktura databáze otázek

otazky		
PK	id	
	tema	text
	otazka	text
	spravna_odpoved	text
	spatna_odpoved1	text
	spatna_odpoved2	text
	spatna_odpoved3	text

Dále má tato třída tři metody: `pridejOtazku`, `otazka` a `pocetOtazek`. Metoda `pridejOtazku` přidává do databáze otázky. Funkce `otazka` v databázi vyhledá otázku se zadaným číslem

a vrátí ji jako list. Metoda `pocetotazek` vrací počet všech otázek v databázi.

Konstruktor třídy `Otazka` se tedy liší od první verze tím, že použije modul `databaze` k vybrání otázky z databáze. Tuto otázku poté rozbalí do několika proměnných.

```
1 import database
2
3 class Otazka(object):
4     def __init__(self, cisloOtazky):
5         self.cislo = cisloOtazky
6         self.otazky = database.Db_otazek()
7         self.celaOtazka = self.otazky.otazka(cisloOtazky)
8         id, self.typ, self.jenOtazka, self.spravnaOdpoved,
           spatnaOdpoved1, spatnaOdpoved2, spatnaOdpoved3 =
           self.celaOtazka
9         self.vsechnyOdpovedi = [self.spravnaOdpoved,
           spatnaOdpoved1, spatnaOdpoved2, spatnaOdpoved3]
```

Zdrojový kód 4.5: Konstruktor třídy `Otazka`

5 Druhá verze

Tato verze se od první výrazně liší. Nejnápadnější z rozdílů je ten, že tato verze má grafické rozhraní. Na vytvoření tohoto grafického rozhraní jsem použila set modulů *Pygame*.

Dalším rozdílem je to, že je určena pouze pro jednoho hráče.

Začala jsem vytvořením třídy `App`. Její konstruktor jako první inicializuje moduly *Pygame*, nastaví proměnnou `_running`, která určuje, zda aplikace běží, na `True`, nadefinuje hlavní plochu a zjistí její velikost. Dále nadefinuje atribut `db` jako databázi otázek.

```
1 class App:
2     def __init__(self):
3         pygame.init()
4         self._running = True
5         self._display_surf = pygame.display.set_mode((0,0),
6             pygame.HWSURFACE | pygame.DOUBLEBUF, pygame.
7             FULLSCREEN)
8         self.sirka, self.vyska = self._display_surf.get_size()
9
10        self.db = database.Db_otazek()
```

Zdrojový kód 5.1: Konstruktor třídy `App`

První metodou třídy `App` je `on_execute`, která obsluhuje hlavní smyčku aplikace. Pokud je proměnná `_running` rovna `True`, tzn. pokud má aplikace běžet, nastaví tato metoda pozadí na bílou barvu a zavolá metody `on_event` a `on_render`. V opačném případě aplikaci zavře.

```
1 def on_execute(self):
2     while self._running:
3         self._display_surf.fill((255,255,255))
4         for event in pygame.event.get():
5             self.on_event(event)
6         self.on_render()
7     pygame.quit()
```

Zdrojový kód 5.2: Metoda `on_execute`

Metoda `on_event` slouží pro obsluhu událostí. Metoda `on_render` rozhoduje, která obrazovka se hráči bude zobrazovat.

Poté jsem vytvořila grafické rozhraní pro pokládání otázek a odpovídání na ně. Byl to zatím jednoduchý návrh, který u horního okraje obrazovky vykreslil otázku a zobrazil 4 možné odpovědi. Když hráč některou z nich zvolil, program mu ukázal, zda je jeho odpověď správná, či nikoli. Na vybírání otázek z databáze jsem využila již zmíněnou třídu `Otazka`. Na vykreslení a funkcionalitu tlačítek jsem použila modul `button`.

Tento modul má třídu `Button`. Tato třída umožňuje vytvoření tlačítka a práci s ním. Má čtyři parametry: `x`, `y`, `obrazek` a `text`. Parametry `x` a `y` udávají souřadnice levého horního rohu tlačítka. Parametr `obrazek`, který slouží pro určení obrázku, který se použije pro vykreslení tlačítka, a parametr `text` udává text, který bude na tlačítku vykreslen. Tento parametr není povinný a jeho výchozí hodnota je prázdný řetězec. Tato třída má dvě metody. První, s názvem `handle_event`, slouží pro zjišťování, zda hráč na tlačítko klikl. Druhá metoda s názvem `zobraz` vykreslí tlačítko na zadanou plochu a poté na toto tlačítko vypíše zadaný text.

```
1 def handle_event(self, event):
2     if event.type == pygame.MOUSEBUTTONDOWN:
3         if self.rect.collidepoint(event.pos):
4             return True
5     return False
```

Zdrojový kód 5.3: Metoda `handle_event` třídy `Button`

Poté jsem přidala hlavní menu. To mělo zatím jen dvě tlačítka: „Hrát“ a „Opustit“. Tato tlačítka byla opět vytvořena pomocí modulu `button`. Tlačítko „Hrát“ zobrazilo hráči otázku a tlačítko „Opustit“ zavřelo aplikaci.

5.1 Verze 2.1 Přidání souboje

Jako další jsem přidala třídu `Tvor` a třídu `Hrac`. Tyto třídy nemají žádné metody a slouží tedy jen pro vytváření instancí tvorů a hráče s různými parametry. Také jsem vytvořila list se všemi druhy tvorů v této hře a přidala ho do proměnné `tvori`.

```

1 self.tvori = [Tvor("pavouk", 50, "obrazky/pavouk.png", 15, 30),
2               Tvor("koza", 60, "obrazky/koza.jpg", 10, 35),
3               Tvor("komár", 20, "obrazky/komar.png", 20, 20),
4               Tvor("netopýr", 55, "obrazky/netopyr.png", 12,
5                   30),
6               Tvor("ryba", 50, "obrazky/ryba.jpg", 10, 25),]

```

Zdrojový kód 5.4: Proměnná `tvori`

Spolu s nimi jsem také přidala metodu `souboj`, která má dva parametry a těmi jsou `vyherce` a `porazeny`. Metoda odečte poškození výherce od životů poraženého. Pokud poražený zemřel, neboli pokud mu životy klesly na nulu nebo níž, vrátí metoda `True`. V opačném případě vybere další otázku a vrátí `False`.

```

1 def souboj(self, vyherce, porazeny):
2     porazeny.zivoty = porazeny.zivoty - vyherce.utok
3     if porazeny.zivoty <= 0:
4         self.zobrazuj_mapu = True
5         self.zobrazuj_otazku = False
6         return True
7     else:
8         self.vyber_otazky()
9         return False

```

Zdrojový kód 5.5: Metoda `souboj`

5.2 Verze 2.2 Přidání mapy

Jako další jsem přidala mapu. Mapa byla původně tvořena čtvercovou sítí o velikosti 20×20 , z čehož mohl hráč vidět při pohybu po mapě 7×7 okolí kolem něho. Nakonec jsem se ale rozhodla použít síť pouze 10×10 a hráči zobrazovat pouze oblast 3×3 . Pohyb po mapě je tak jasnější, obzvláště díky větším políčkům. Mapa je definována v proměnné `mapa` třídy `App` jako slovník, ve kterém je ke každé souřadnici přiřazen typ políčka.

```

1 self.mapa = {(1,1): "L", (2,1): "L", (3,1): "L", (4,1): "V", (5,1):
2             "V", (6,1): "L", (7,1): "L", (8,1): "L", (9,1): "L", (10,1): "L",
3             (1,2): "L", (2,2): "L", (3,2): "V", (4,2): "V", (5,2): "O", (6,2):
4             "L", (7,2): "J", (8,2): "C", (9,2): "L", (10,2): "L",
5             ...
6             (1,10): "V", (2,10): "L", (3,10): "J", (4,10): "H", (5,10): "H", (6,10):
7             "H", (7,10): "H", (8,10): "H", (9,10): "H", (10,10): "H",}

```

Zdrojový kód 5.6: Proměnná `mapa`

5.3 Verze 2.3 Přidání obchodu

V této malé verzi jsem přidala obchod a třídu `Predmet`. Tato třída má tři parametry a těmi jsou `nazev`, `obrazek`, `cena`. Tato třída má jednu metodu `pouzit`. Tato metoda buď přidá hráči pět poškození, pokud se předmět jmenuje *Lektvar síly*, anebo hráče vyléčí a přidá 20 životů, pokud se jmenuje *Lektvar zdraví*.

```
1 def pouzit(self):
2     if self.nazev == "Lektvar síly":
3         hrac.utok = hrac.utok + 5
4     elif self.nazev == "Lektvar zdraví":
5         hrac.zivoty = hrac.max_zivoty
6         hrac.max_zivoty = hrac.max_zivoty + 20
```

Zdrojový kód 5.7: Metoda `pouzit`

Když je hráč v obchodě, vidí dvě tlačítka, každé sloužící k nákupu jednoho z lektvarů. Dále se hráči zobrazuje, kolik zlatáků mu zbývá. V levém horním rohu obrazovky se nachází tlačítko „Zpět“, díky kterému se může hráč dostat zpět na mapu.

5.4 Verze 2.4 Přidání možnosti vložit otázku

Poté jsem přidala možnost přidat otázku do databáze. Aby mohl uživatel zadat všechny části otázky, bylo nutné vytvořit třídu pro práci s textovými poli. Tuto třídu jsem přidala do modulu `button`. Tuto třídu jsem pojmenovala `Textbox`. Na rozdíl od třídy `Button` má pouze tři parametry. Chybí jí parametr `text`, ostatní jsou stejné. Má také metody `handle_event` a `zobraz`. Metoda `zobraz` je stejná jako u třídy `Button`, metoda `handle_event` se ale liší tím, že zjišťuje, zda má být textové pole aktivní, a pokud ano, ukládá, co uživatel píše, do proměnné `text`. Textové pole je aktivní, pokud na něj uživatel klikne. Jakmile ale klikne mimo něj, přestává být aktivní.

```
1 def handle_event(self, event):
2     if event.type == pygame.MOUSEBUTTONDOWN:
3         if self.rect.collidepoint(event.pos):
4             self.aktivni = True
5         else:
6             self.aktivni = False
7
```



```

8     if event.type == pygame.TEXTINPUT and self.aktivni:
9         self.text += event.text
10
11    if event.type == pygame.KEYDOWN and self.aktivni:
12        if event.key == pygame.K_BACKSPACE:
13            self.text = self.text[:-1]

```

Zdrojový kód 5.8: Metoda `handle_event` třídy `Textbox`

Na hlavní menu jsem tedy přidala tlačítko „Přidat otázku“, na které když hráč klikne, zobrazí se mu rozhraní pro přidání otázky. Toto rozhraní obsahuje šest textových polí vytvořených pomocí třídy `Textbox`, každé pro jednu část otázky. Dále je na obrazovce tlačítko „Zpět“, které vrátí hráče na hlavní menu a tlačítko „Přidat otázku“, které přidá zadanou otázku do databáze.

5.5 Verze 2.5 Rozdělení otázek tvorům podle témat

Nakonec jsem se rozhodla rozdělit otázky jednotlivým druhům tvorů podle tématu. Takto může hráč lépe předpokládat obor, ze kterého otázka bude a může si tak některá témata cíleněji procvičit. Abych toto mohla udělat, musela jsem nejdříve vytvořit další dvě metody třídy `Db_otazek` v modulu `databaze`. Jedna z těchto metod se jmenuje `vsechna_temata` a vrací všechna témata otázek, která se v databázi vyskytují. Druhá metoda se jmenuje `tema` a vrací list čísel všech otázek, které mají určité téma. Poté jsem přidala parametr `otazky` třídy `Tvor`, jehož výchozí hodnota je prázdný list. V konstruktoru třídy `App` se k tomuto listu připojí listy otázek jednotlivých témat, která budou k tomuto tvorů přiřazena. Témata se tvorům přiřazují automaticky a to podle zbytkových tříd po celočíselném dělení pořadí tématu počtem všech tvorů.

```

1 vsechna_temata = self.db.vsechna_temata()
2 for i in range(len(vsechna_temata)):
3     self.tvori[i%len(self.tvori)].otazky = self.tvori[i%len(
        self.tvori)].otazky + self.db.tema(vsechna_temata[i])

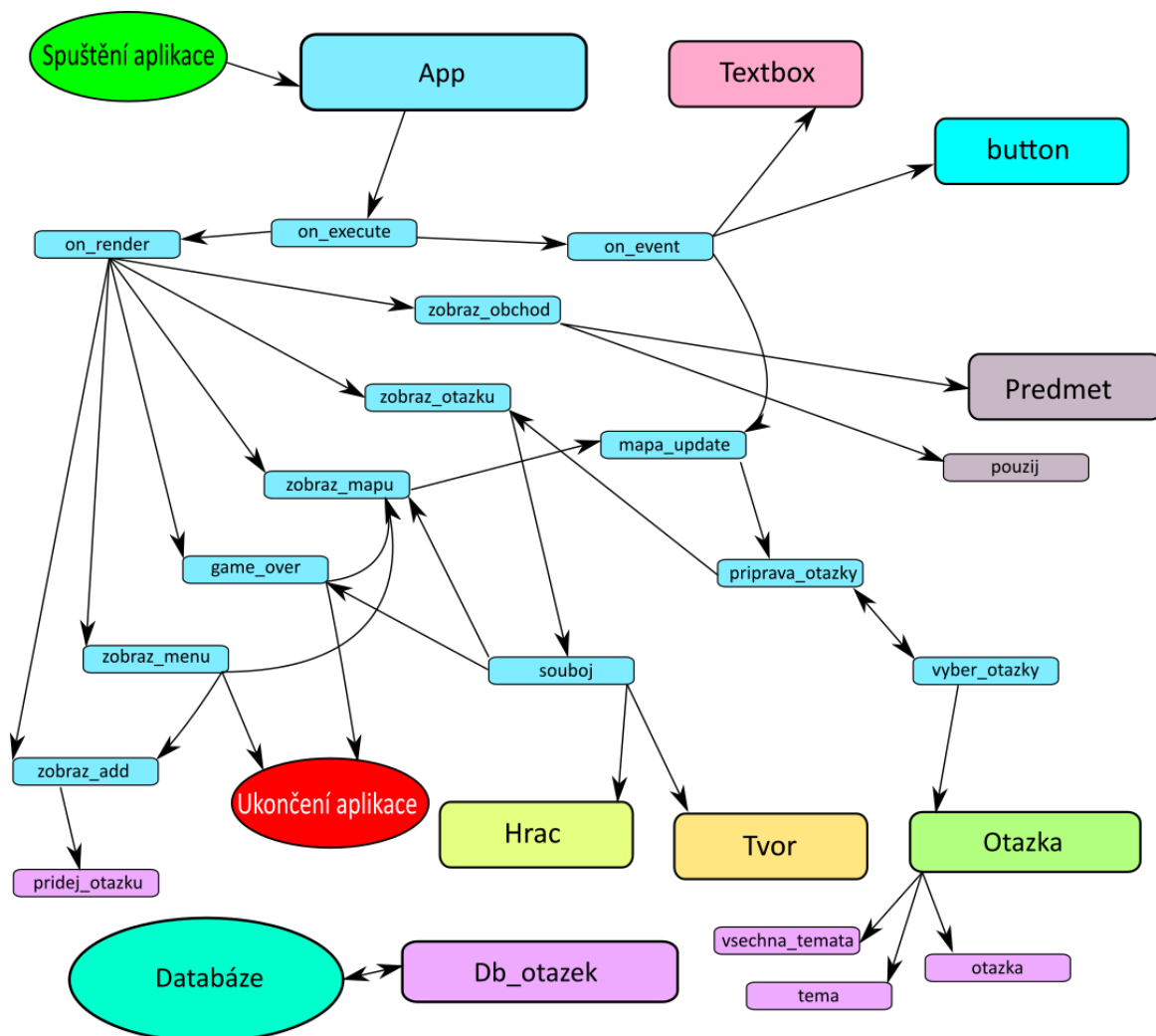
```

Zdrojový kód 5.9: Přiřazení otázek tvorům

6.1 Blokové schéma

6.1 Blokové schéma

Toto schéma slouží k vizualizaci postupu programu od spuštění aplikace po její ukončení. Každý velký blok zobrazuje jednu ze tříd tohoto programu a menší bloky zobrazují vybrané metody těchto tříd. Blok třídy a všechny její vyobrazené metody mají vždy stejnou barvu.



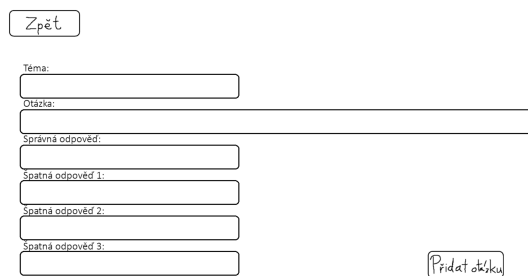
Obrázek 6.1: Blokové schéma

6.2 Průběh hry

Když uživatel hru spustí, jako první se mu zobrazí hlavní menu. To obsahuje tři tlačítka. Spodní tlačítko s nápisem „Ukončit“ zavře aplikaci. Prostřední tlačítko s nápisem „Přidat otázku“ otevře rozhraní, pomocí něhož může hráč zadat všechny části otázky a přidat ji do databáze. Poslední, horní, tlačítko s nápisem „Hrát“ spustí samotnou hru.

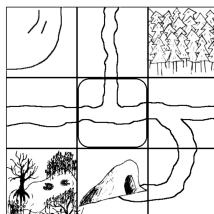


Obrázek 6.2: Hlavní menu

Rozhraní pro přidávání otázek. V horní části je tlačítko „Zpět“. Níže jsou pole pro zadání: „Téma:“, „Otázka:“, „Správná odpověď:“, „Špatná odpověď 1:“, „Špatná odpověď 2:“ a „Špatná odpověď 3:“. Vpravo dole je tlačítko „Přidat otázku“.

Obrázek 6.3: Rozhraní pro přidávání otázek

Když spustí hru, vidí hráč mapu. Po té se může pohybovat pomocí kláves W,A,S a D. Mapa je tvořena čtvercovými políčky, které jsou uspořádány v síti o velikosti 10×10. Každé políčko má přiřazený jeden ze sedmi typů. Těmito typy jsou: obchod, cesta, les, bažina, hory, jeskyně a voda. Hráč se může volně pohybovat po cestách, ale jakmile odbočí, zaútočí na něj nějaký tvor. Tvorů je celkem pět druhů (pavouk, ryba, komár, netopýr a koza) a ke každému typu políčka je přiřazen jiný druh (les – pavouk, voda – ryba, bažina – komár, jeskyně – netopýr, hory – koza). Když tedy hráč vejde na jiné políčko než cesta nebo obchod, zaútočí na něj příslušný tvor. S tímto tvorem se poté hráč musí utkat ve vědomostním souboji.

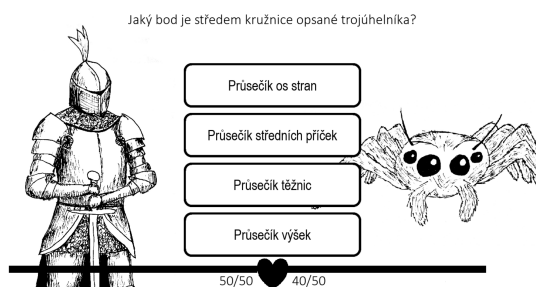


Pro pohyb použij W,A,S,D

Obrázek 6.4: Herní mapa

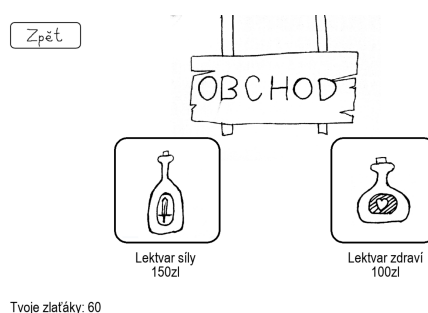
V horní části obrazovky se hráči zobrazí otázka a pod ní čtyři tlačítka s možnými od-

pověďmi. Nalevo od těchto tlačítek je vyobrazen hráč a pod ním ukazatel zdraví. Na pravé straně je poté vykreslen tvor, který hráče napadl. Když hráč správně odpoví na otázku, dá poškození tvorovi. Když ale naopak odpoví špatně, dostane sám poškození. Pokud se hráči podaří zabít tvora, přičte se mu skóre a zlatáky.



Obrázek 6.5: Souboj

Zlatáky jsou herní měna, kterou může hráč utratit v obchodě. Když hráč vejde do obchodu, zobrazí se mu v horní části obrazovky tlačítko zpět, pomocí něhož se může hráč vrátit zpět na mapu. Ve spodní části obrazovky je napsáno, kolik zlatáků má hráč. Dále se zde zobrazují dvě tlačítka, pomocí nichž může hráč koupit jeden ze dvou lektvarů. *Lektvar zdraví* doplní hráči ztracené životy a přidá mu 20 životů navíc. *Lektvar síly* hráči zvýší poškození, které dává tvorům. Efekt lektvarů začne být aktivní, jakmile si ho hráč koupí.



Obrázek 6.6: Obchod

Hra také počítá celkové skóre hráče a čím je vyšší, tím náročnější je hra. Každému tvorovi se k počátečnímu poškození přidá setina hráčova skóre a k životům se mu přičte desetina hráčova skóre.

Pokud v nějakém souboji hráč zemře, zobrazí se mu na obrazovce zpráva „Zemřel jsi“, celkové dosažené skóre a dvě tlačítka. Pomocí jednoho může hráč hru ukončit, pomocí druhého

hrát znovu. Hráči se vynuluje skóre a zlatáky a hráči i tvorům se vrátí poškození a životy na počáteční hodnoty.



Obrázek 6.7: Konec hry

6.3 Grafické rozhraní

Rozhodla jsem se pro jednoduchý, černobílý vzhled. Všechny obrázky použité v této hře jsem kreslila ručně na papír, poté skenovala a upravovala v programu Malování. V celé hře jsem volila dobře čitelná, bezpatková písma. Pro všechna tlačítka jsem využila písmo Arial a pro zobrazení otázky písmo Calibri, protože písmo Arial bylo moc široké a některé otázky by se nevešly na obrazovku.

Závěr

První část práce čtenáři představila současnou scénu vědomostních her. Zaměřila se i na jejich vliv na výsledky vzdělávání. Bylo zjištěno, že videohry mohou zvýšit motivaci studentů a jejich zájem o výuku.

Dále práce představila pojem *agilní metoda* a některé konkrétní takové metody. Při analýze dostupných zdrojů bylo zjištěno, že nelze určit, zda jsou tyto agilní metody efektivnější než klasické postupy. Podle těchto zdrojů jsou však názory vývojářů i zákazníků na tyto metody lepší než na klasické postupy.

V hlavní části jsem popsala postup při vývoji vlastní vědomostní hry. Vytvořila jsem nejdříve jednoduchý a obecný návrh a ten jsem poté pomocí malých verzí vylepšovala a upřesňovala. Z tohoto faktu je možno usoudit, že jsem postupovala agilně. Podařilo se mi vytvořit funkční vědomostní hru, do které si může hráč přidat vlastní otázky. Hráč si při hraní může také cíleně procvičovat jen některá témata. Hra také motivuje hráče ve hře postupovat počítáním skóre, které se při konci hry zobrazí.

Hra by ale mohla být více přehledná. V současném stavu může působit zmatečně, obzvlášť pro hráče, kteří často videohry nehrají. S přehledností by mohly pomoci vysvětlivky jednotlivých aspektů hry, jako je mapa nebo ukazatele zdraví. Dále by pro přehlednost mohly být přínosné animace při přechodu mezi jednotlivými obrazovkami, které by lépe vysvětlily, co se zrovna stalo (např. napadení hráče tvorem nebo poražení tvora). Dále by hře mohlo prospět, kdyby si hráč mohl zobrazit celou databázi otázek a přímo s ní interagovat. Například by mohl hráč přidat několik otázek zároveň, vybrat si, které se budou zobrazovat nebo některé smazat.

Díky agilnímu postupu a objektově orientované povaze kódu by ale bylo poměrně jednoduché hru v budoucnu vylepšit a doplnit ji o novou funkcionalitu.

Bibliografie

1. *About us* [online]. 2025. [cit. 2026-01-12]. Dostupné z: <https://www.sporcle.com/about/>.
2. AGILEALLIANCE. *A Short History of Agile* [online]. 2025. [cit. 2026-01-28]. Dostupné z: <https://agilealliance.org/a-short-history-of-agile/>.
3. AGILEALLIANCE. *AGILE 101: What is Agile?* [online]. 2025. [cit. 2026-01-24]. Dostupné z: <https://agilealliance.org/agile101/>.
4. ARNOLD, Brian; KOEHLER, Matthew; GREENHALGH, Spencer. Design Principles for Creating and Maintaining Immersive Experiences in Educational Games. In: 2016.
5. AUCCOIN, Don. *On top of the world* [online]. 2009. [cit. 2025-12-16]. Dostupné z: https://web.archive.org/web/20100824073333/http://www.boston.com/lifestyle/articles/2009/06/16/visitors_to_website_sporcle_learn_geography_and_other_subjects___and_have_fun.
6. BECK, Kent. *Extrémní programování*. Praha: Grada, 2002. ISBN 80-247-0300-9.
7. BECK, Kent. *Test-driven development: By example*. 20. printing. Addison-Wesley, 2015. ISBN 0321146530.
8. BECK, Kent; GRENNING, James; MARTIN, Robert C.; BEEDLE, Mike; HIGSMITH, Jim; MELLOR, Steve; BENNEKUM, Arie van; HUNT, Andrew; SCHWABER, Ken; COCKBURN, Alistair; JEFFRIES, Ron; SUTHERLAND, Jeff; CUNNINGHAM, Ward; KERN, Jon; THOMAS, Dave; FOWLER, Martin; MARICK, Brian. *Manifest Agilního vývoje software* [online]. 2001. [cit. 2026-01-20]. Dostupné z: <https://agilemanifesto.org/iso/cs/manifesto.html>.
9. BECK, Kent; GRENNING, James; MARTIN, Robert C.; BEEDLE, Mike; HIGSMITH, Jim; MELLOR, Steve; BENNEKUM, Arie van; HUNT, Andrew; SCHWABER, Ken; COCKBURN, Alistair; JEFFRIES, Ron; SUTHERLAND, Jeff; CUNNINGHAM,

- Ward; KERN, Jon; THOMAS, Dave; FOWLER, Martin; MARICK, Brian. *Principy stojící za Agilním Manifestem* [online]. 2001. [cit. 2026-01-25]. Dostupné z: <https://agilemanifesto.org/iso/cs/principles.html>.
10. *Diskuze Dobyvatel* [online]. 2016. [cit. 2026-01-17]. Dostupné z: <https://zpovednice.eu/detail.php?statusik=950876>.
 11. *Dobyvatel* [online]. 2023. [cit. 2026-01-17]. Dostupné z: <https://web.archive.org/web/20230623084926/https://play.google.com/store/apps/details?id=com.thxgames.triviador&hl=cs>.
 12. *Dobyvatel* [online]. 2025. [cit. 2025-11-11]. Dostupné z: <https://play.google.com/store/apps/details?id=com.thxgames.triviador&hl=cs&pli=1>.
 13. *Dobyvatel (1.0)* [online]. 2013. [cit. 2025-12-15]. Dostupné z: <https://mujsoubor.cz/prohlizecove-hry/dobyvatel>.
 14. DOMBROVSKÝ, Jiří. *Petice za obnovení staré verze hry Dobyvatel* [online]. 2016. [cit. 2025-11-11]. Dostupné z: <https://e-petice.cz/petitions/petice-za-obnoveni-hry-dobyvatel.html>.
 15. DONDLINGER, Mary. Educational Video Game Design: A Review of the Literature. *Journal of Applied Educational Technology*. 2007, roč. 4.
 16. DYBÅ, Tore; DINGSØYR, Torgeir. Empirical studies of agile software development: A systematic review. *Information and Software Technology*. 2008, roč. 50, č. 9–10, s. 833–859. ISSN 0950-5849. Dostupné z DOI: 10.1016/j.infsof.2008.01.006.
 17. GOOGLE. *Hodnocení a recenze v Obchodě Play* [online]. 2026. [cit. 2026-01-17]. Dostupné z: <https://play.google/comment-posting-policy/?hl=cs>.
 18. HANNAY, Jo E.; DYBÅ, Tore; ARISHOLM, Erik; SJØBERG, Dag I.K. The effectiveness of pair programming: A meta-analysis. *Information and Software Technology*. 2009, roč. 51, č. 7, s. 1110–1122. ISSN 0950-5849. Dostupné z DOI: 10.1016/j.infsof.2009.02.001.
 19. HANOA, Eilert. *Kahoot! delivers record growth with big global shift to remote learning* [online]. 2020. [cit. 2025-12-01]. Dostupné z: <https://kahoot.com/blog/2020/07/02/kahoot-delivers-record-growth-global-shift-remote-learning>.

20. HARVEY, Kerric. *Encyclopedia of social media and politics*. SAGE Reference, 2014. ISBN 9781452290263.
21. ILIEVA, Sonia; IVANOV, P.; STEFANOVA, Eliza. Analyses of an agile methodology implementation. In: 2004, sv. 30, s. 326–333. ISBN 0-7695-2199-1. Dostupné z DOI: 10.1109/EURMIC.2004.1333387.
22. KAHOOT! *About us* [online]. 2025. [cit. 2025-11-30]. Dostupné z: <https://kahoot.com/company>.
23. KAHOOT! *What is Kahoot!?* [online]. 2025. [cit. 2025-12-01]. Dostupné z: <https://kahoot.com/what-is-kahoot/>.
24. KEANE, Jonathan. *Norwegian edtech company Kahoot! reaches 1 billion players, 40 million MAUs* [online]. 2017. [cit. 2025-12-01]. Dostupné z: <https://tech.eu/2017/03/06/kahoot-1-billion-players>.
25. MACÍCH, Jiří. *Nova spustila online hru Dobyvatel.cz, mutaci strategie Conquistador* [online]. 2010. [cit. 2025-11-10]. Dostupné z: <https://www.lupa.cz/clanky/nova-spustila-online-hru-dobyvatel-cz>.
26. MANN, C.; MAURER, Frank. A case study on the impact of scrum on overtime and customer satisfaction. In: 2005, sv. 2005, s. 70 –79. ISBN 0-7695-2487-7. Dostupné z DOI: 10.1109/ADC.2005.1.
27. MEČIAR, Maroš. *Chyběl mi starý Dobyvatel, tak jsem vytvořil vlastní hru* [online]. 2025. [cit. 2026-01-16]. Dostupné z: <https://www.youtube.com/watch?v=bHFS34n-gkY&t=105s>.
28. MEČIAR, Maroš. *Vyzyvatel.com* [online]. 2025. [cit. 2026-01-17]. Dostupné z: <https://vyzyvatel.com>.
29. *O hře* [online]. 2017. [cit. 2025-11-10]. Dostupné z: <https://web.archive.org/web/20170617023135/https://thxgames.zendesk.com/hc/cs/articles/207966549-0-he>.
30. *Oficiální pravidla hry* [online]. 2018-10. [cit. 2025-11-10]. Dostupné z: <https://web.archive.org/web/20181031133118/https://thxgames.zendesk.com/hc/cs/articles/207970149-Pravidla-hry>.

31. PYGAME. *About - Wiki* [online]. 2026. [cit. 2026-01-11]. Dostupné z: <https://www.pygame.org/wiki/about>.
32. PYGAME. *GettingStarted — wiki* [online]. 2026. [cit. 2026-01-11]. Dostupné z: <https://www.pygame.org/wiki/GettingStarted>.
33. RAVIPATI, Sri. *Homework Game Changer Kahoot! Launches Mobile App* [online]. 2017. [cit. 2025-12-09]. Dostupné z: <https://thejournal.com/articles/2017/09/14/homework-game-changer-kahoot-launches-mobile-app.aspx>.
34. SCHWABER, Ken; SUTHERLAND, Jeff. *The Scrum Guide*. 2020.
35. SPORCLE. *A Brief History of Sporcle: The First 10 Years* [online]. 2017. [cit. 2026-01-13]. Dostupné z: <https://www.sporcle.com/blog/2017/01/what-is-sporcle-a-brief-history/>.
36. SPORCLE. *Find the US States - No Outlines Minefield* [online]. 2026. [cit. 2026-01-12]. Dostupné z: <https://www.sporcle.com/games/mhershfield/us-states-no-outlines-minefield>.
37. SPORCLE. *Popular: All categories* [online]. 2026. [cit. 2026-01-12]. Dostupné z: <https://www.sporcle.com/popular/alltime>.
38. SPORCLE. *Why We Changed the Logo (Again)* [online]. 2022. [cit. 2026-01-12]. Dostupné z: <https://www.sporcle.com/blog/2022/06/why-we-changed-the-logo-again/>.
39. *Sporcle Quiz Categories* [online]. 2025. [cit. 2026-01-12]. Dostupné z: <https://www.sporcle.com/categories/>.
40. THXGAMES. *Game mode: Short Campaign* [online]. 2023. [cit. 2025-11-10]. Dostupné z: <https://help.triviador.com/en/support/solutions/articles/66000215532-game-mode-short-campaign>.
41. THXGAMES. *Nová verze Dobývatele je dostupná pouze na mobilních zařízeních* [online]. 2021. [cit. 2025-11-11]. Dostupné z: <https://web.archive.org/web/20211121231019/https://dobyvatel.triviador.net>.
42. TURAN, Zeynep; MERAL, Elif. Game-Based versus to Non-Game-Based: The Impact of Student Response Systems on Students' Achievements, Engagements and Test Anxieties. *Informatics in Education*. 2018, roč. 17, č. 1, s. 105–116.

43. TURHAN, Burak; LAYMAN, Lucas; DIEP, Madeline; SHULL, Forrest; ERDOGMUS, Hakan. How Effective is Test Driven Development. In: 2010.
44. TVNOVA. *Dobyvatel.cz slaví dva roky od svého spuštění!* [online]. 2012. [cit. 2025-11-10]. Dostupné z: <https://tn.nova.cz/zpravodajstvi/clanek/251791-dobyvatel-cz-slavi-dva-roky-od-sveho-spusteni>.
45. WANG, Alf. The wear out effect of a game-based student response system. *Computers and Education*. 2015, roč. 82, s. 217–227. Dostupné z DOI: 10.1016/j.compedu.2014.11.004.
46. WANG, Alf Inge; LIEBEROTH, Andreas. The effect of points and audio on concentration, engagement, enjoyment, learning, motivation, and classroom dynamics using Kahoot. In: *European conference on games based learning*. Academic conferences international limited, 2016, sv. 20, s. 738–746.
47. WANG, Alf Inge; ØFSDAHL, Terje; MØRCH-STORSTEIN, Ole Kristian. Lecture quiz-a mobile game concept for lectures. In: *Proceedings of the 11th IASTED International Conference on Software Engineering and Application (SEA'07)*. 2007, s. 305–310.
48. WANG, Alf Inge; TAHIR, Rabail. The effect of using Kahoot! for learning – A literature review. *Computers and Education*. 2020, roč. 149, s. 103818. ISSN 0360-1315. Dostupné z DOI: <https://doi.org/10.1016/j.compedu.2020.103818>.
49. WOODFORD, Antonia. *Sporcle ranks colleges based on use* [online]. 2011. [cit. 2025-12-16]. Dostupné z: <https://yaledailynews.com/blog/2011/02/18/sporcle-ranks-colleges-based-on-use/>.
50. WU, Bian; WANG, Alf Inge; BØRRESEN, Erling Andreas; TIDEMANN, Knut Andre. IMPROVEMENT OF A LECTURE GAME CONCEPT - Implementing Lecture Quiz 2.0. In: *Proceedings of the 3rd International Conference on Computer Supported Education*. SciTePress - Science, and Technology Publications, 2011, s. 26–35. Dostupné z DOI: 10.5220/0003331300260035.

Seznam obrázků

1.1	Ukázka GUI „starého“ <i>Dobyvatele</i> [13]	4
1.2	Ukázka GUI „nového“ <i>Dobyvatele</i> [27]	4
1.3	Ukázka GUI mobilní aplikace <i>Dobyvatel</i> [12]	5
1.4	Ukázka GUI hry <i>Vyzyvatel</i> [28]	5
1.5	Současné logo <i>Sporcle</i> [38]	7
1.6	Najdi státy USA – bez obrysů, minové pole[36]	7
6.1	Blokové schéma	28
6.2	Hlavní menu	29
6.3	Rozhraní pro přidávání otázek	29
6.4	Herní mapa	29
6.5	Souboj	30
6.6	Obchod	30
6.7	Konec hry	31

Přílohy

Seznam souborů v přiloženém ZIP archivu

1. `button.py` — modul sloužící pro práci s tlačítky a textovými poly
2. `database.py` — modul sloužící pro komunikaci s databází
3. `main.py` — vykreslování a logika hry
4. `otazky.db` — databáze otázek
5. `otazky.py` — modul sloužící pro tvorbu otázek
6. složka `obrazky` — Obsahuje obrázky využívané ve hře

Aplikaci je možné spustit v příkazovém řádku zadáním příkazů:

```
cd cesta\do\složky\obsahující\program  
python main.py
```